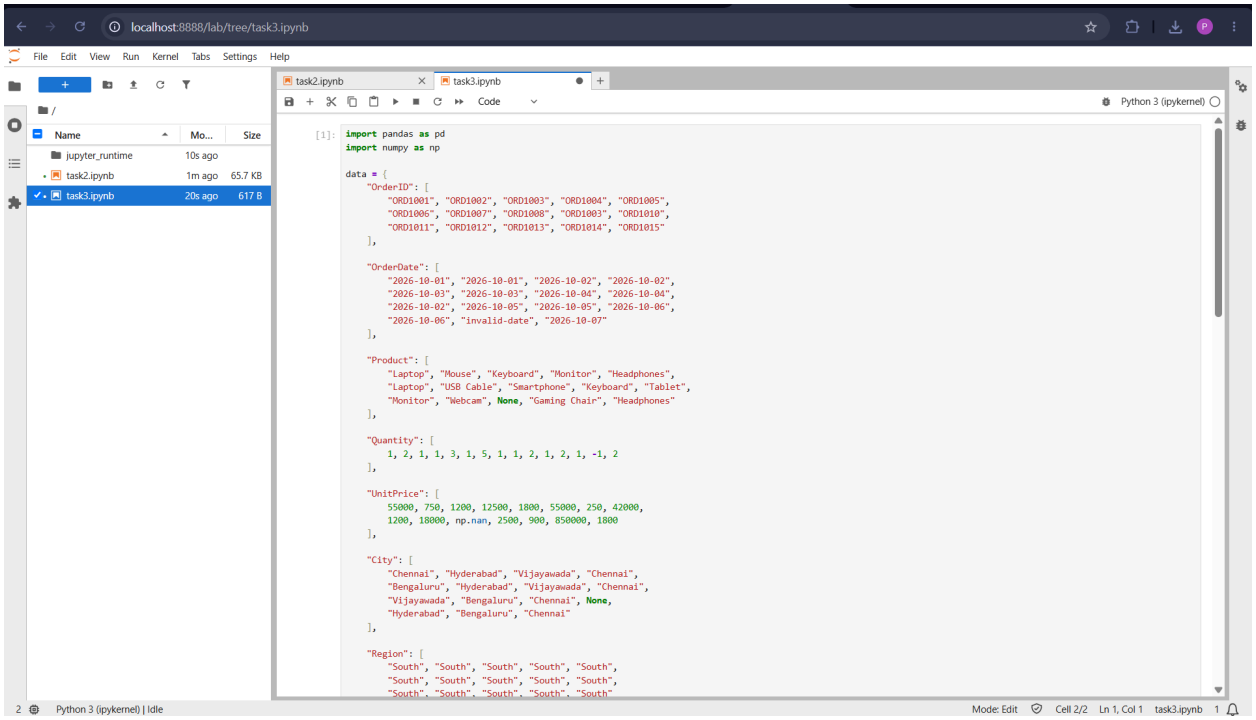

localhost:8888/lab/tree/task3.ipynb
File Edit View Run Kernel Tabs Settings Help
task2.ipynb
task3.ipynb
Code
Python 3 (ipykernel)
Name
Mo...
Size
jupyter_runtime 10s ago
task2.ipynb 1m ago 65.7 KB
task3.ipynb 20s ago 617 B
[1]:
import pandas as pd
import numpy as np
data = {
"OrderID": [
"ORD1001", "ORD1002", "ORD1003", "ORD1004", "ORD1005",
"ORD1006", "ORD1007", "ORD1008", "ORD1003", "ORD1010",
"ORD1011", "ORD1012", "ORD1013", "ORD1014", "ORD1015"
],
"OrderDate": [
"2026-10-01", "2026-10-01", "2026-10-02", "2026-10-02",
"2026-10-03", "2026-10-03", "2026-10-04", "2026-10-04",
"2026-10-02", "2026-10-05", "2026-10-05", "2026-10-06",
"2026-10-06", "invalid-date", "2026-10-07"
],
"Product": [
"Laptop", "Mouse", "Keyboard", "Monitor", "Headphones",
"Laptop", "USB Cable", "Smartphone", "Keyboard", "Tablet",
"Monitor", "Webcam", None, "Gaming Chair", "Headphones"
],
"Quantity": [
1, 2, 1, 1, 3, 1, 5, 1, 1, 2, 1, 2, 1, -1, 2
],
"UnitPrice": [
55000, 750, 1200, 12500, 1800, 55000, 250, 42000,
1200, 18000, np.nan, 2500, 900, 850000, 1800
],
"City": [
"Chennai", "Hyderabad", "Vijayawada", "Chennai",
"Bengaluru", "Hyderabad", "Vijayawada", "Chennai",
"Vijayawada", "Bengaluru", "Chennai", None,
"Hyderabad", "Bengaluru", "Chennai"
],
"Region": [
"South", "South", "South", "South", "South",
"South", "South", "South", "South", "South",
"South", "South", "South", "South", "South"
]
}
df = pd.DataFrame(data)
Calculate revenue
df["Revenue"] = df["Quantity"] * df["UnitPrice"]
df
[1]:
OrderID OrderDate Product Quantity UnitPrice City Region Revenue
0 ORD1001 2026-10-01 Laptop 1 55000.0 Chennai South 55000.0
1 ORD1002 2026-10-01 Mouse 2 750.0 Hyderabad South 1500.0
2 ORD1003 2026-10-02 Keyboard 1 1200.0 Vijayawada South 1200.0
3 ORD1004 2026-10-02 Monitor 1 12500.0 Chennai South 12500.0
4 ORD1005 2026-10-03 Headphones 3 1800.0 Bengaluru South 5400.0
5 ORD1006 2026-10-03 Laptop 1 55000.0 Hyderabad South 55000.0
6 ORD1007 2026-10-04 USB Cable 5 250.0 Vijayawada South 1250.0
7 ORD1008 2026-10-04 Smartphone 1 42000.0 Chennai South 42000.0
8 ORD1003 2026-10-02 Keyboard 1 1200.0 Vijayawada South 1200.0
9 ORD1010 2026-10-05 Tablet 2 18000.0 Bengaluru South 36000.0
10 ORD1011 2026-10-05 Monitor 1 NaN Chennai South NaN
11 ORD1012 2026-10-06 Webcam 2 2500.0 NaN South 5000.0
12 ORD1013 2026-10-06 NaN 1 900.0 Hyderabad South 900.0
13 ORD1014 invalid-date Gaming Chair -1 850000.0 Bengaluru South -850000.0
14 ORD1015 2026-10-07 Headphones 2 1800.0 Chennai South 3600.0
Python 3 (ipykernel) | Idle
Mode: Edit
Cell 2/2
Ln 1, Col 1
task3.ipynb

Identify Missing Values

[2]:
```
df
```

[2]:

	OrderID	OrderDate	Product	Quantity	UnitPrice	City	Region	Revenue
0	ORD1001	2026-10-01	Laptop	1	55000.0	Chennai	South	55000.0
1	ORD1002	2026-10-01	Mouse	2	750.0	Hyderabad	South	1500.0
2	ORD1003	2026-10-02	Keyboard	1	1200.0	Vijayawada	South	1200.0
3	ORD1004	2026-10-02	Monitor	1	12500.0	Chennai	South	12500.0
4	ORD1005	2026-10-03	Headphones	3	1800.0	Bengaluru	South	5400.0
5	ORD1006	2026-10-03	Laptop	1	55000.0	Hyderabad	South	55000.0
6	ORD1007	2026-10-04	USB Cable	5	250.0	Vijayawada	South	1250.0
7	ORD1008	2026-10-04	Smartphone	1	42000.0	Chennai	South	42000.0
8	ORD1003	2026-10-02	Keyboard	1	1200.0	Vijayawada	South	1200.0
9	ORD1010	2026-10-05	Tablet	2	18000.0	Bengaluru	South	36000.0
10	ORD1011	2026-10-05	Monitor	1	NaN	Chennai	South	NaN
11	ORD1012	2026-10-06	Webcam	2	2500.0	NaN	South	5000.0
12	ORD1013	2026-10-06	NaN	1	900.0	Hyderabad	South	900.0
13	ORD1014	invalid-date	Gaming Chair	-1	850000.0	Bengaluru	South	-850000.0
14	ORD1015	2026-10-07	Headphones	2	1800.0	Chennai	South	3600.0

[3]:
```
df.isnull()
```

[3]:

	OrderID	OrderDate	Product	Quantity	UnitPrice	City	Region	Revenue
0	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False
5	False	False	False	False	False	False	False	False
6	False	False	False	False	False	False	False	False
7	False	False	False	False	False	False	False	False
8	False	False	False	False	False	False	False	False
9	False	False	False	False	False	False	False	False
10	False	False	False	False	True	False	False	True
11	False	False	False	False	False	True	False	False
12	False	False	True	False	False	False	False	False
13	False	False	False	False	False	False	False	False
14	False	False	False	False	False	False	False	False

[4]:
```
df.isnull().sum()
```

[4]:
```
OrderID      0
OrderDate    0
Product      1
Quantity     0
UnitPrice    1
City         1
Region       0
Revenue      1
dtype: int64
```

```
[5]: df[df.isnull().any(axis=1)]
```

[5]:

	OrderID	OrderDate	Product	Quantity	UnitPrice	City	Region	Revenue
10	ORD1011	2026-10-05	Monitor	1	NaN	Chennai	South	NaN
11	ORD1012	2026-10-06	Webcam	2	2500.0	NaN	South	5000.0
12	ORD1013	2026-10-06	NaN	1	900.0	Hyderabad	South	900.0

```
[6]: df["OrderDate"] = pd.to_datetime(
         df["OrderDate"],
         errors="coerce"
     )
```

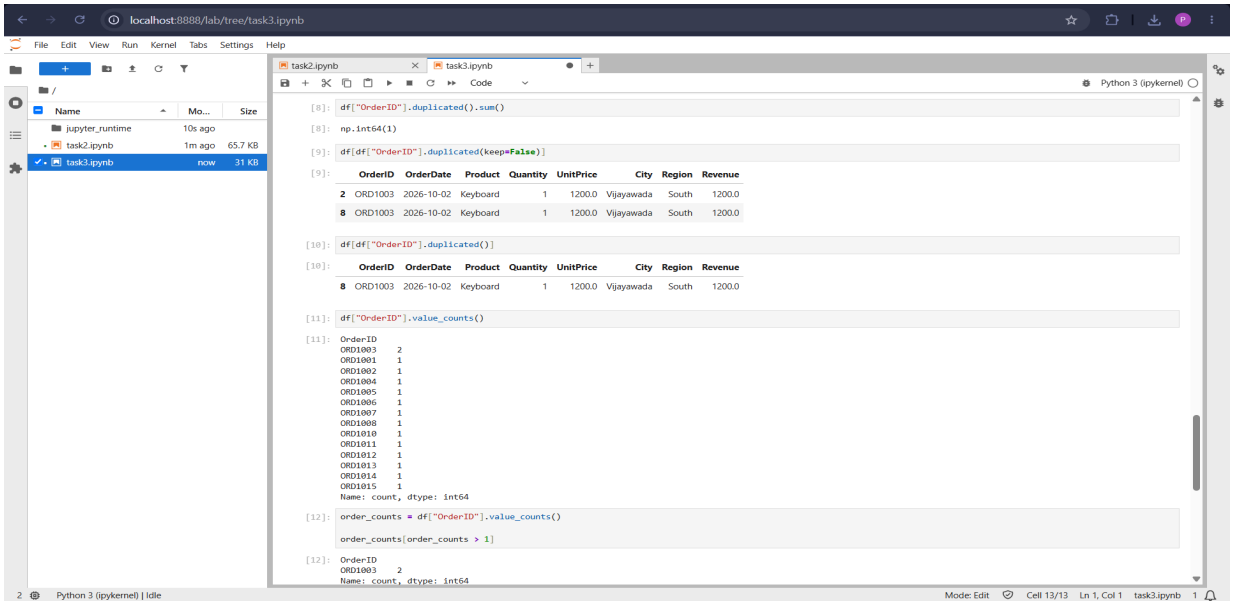
```
[7]: df.isnull().sum()
```

```
[7]: OrderID      0
     OrderDate    1
     Product      1
     Quantity     0
     UnitPrice    1
     City         1
     Region       0
     Revenue      1
     dtype: int64
```

Possible causes: Incomplete data entry, missing customer information, or invalid date formats.

Solutions: Verify the original records, recover missing information where possible, and validate dates before saving them.

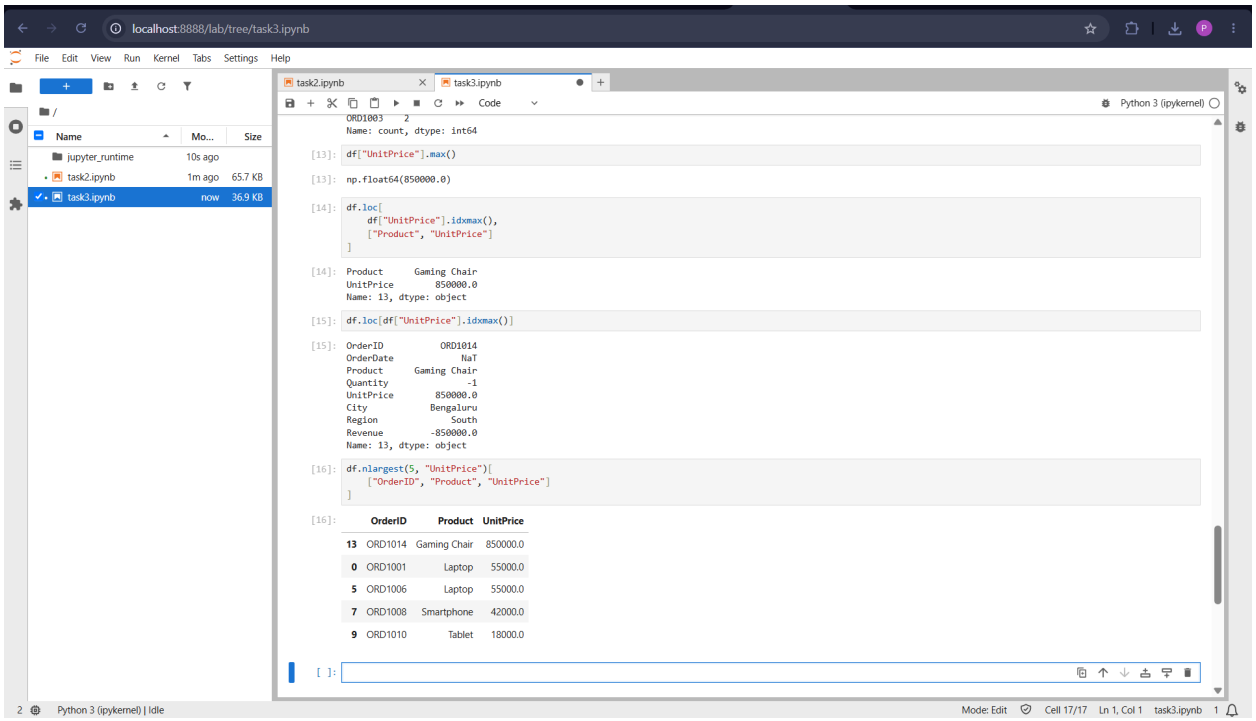
Identify Duplicate Order IDs



Possible causes: Duplicate data imports, repeated submissions, or incorrect order ID generation.

Solutions: Verify the source records and remove duplicates only after confirming that they represent the same order.

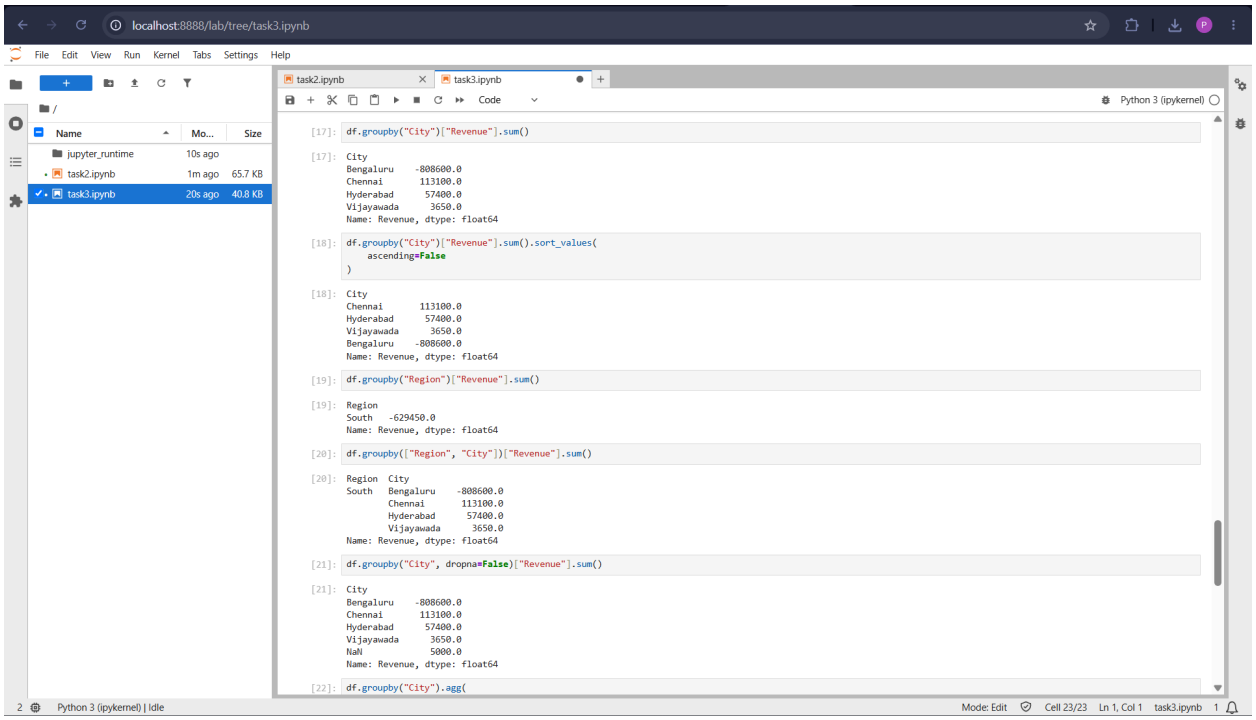
Find the Highest-Priced Product

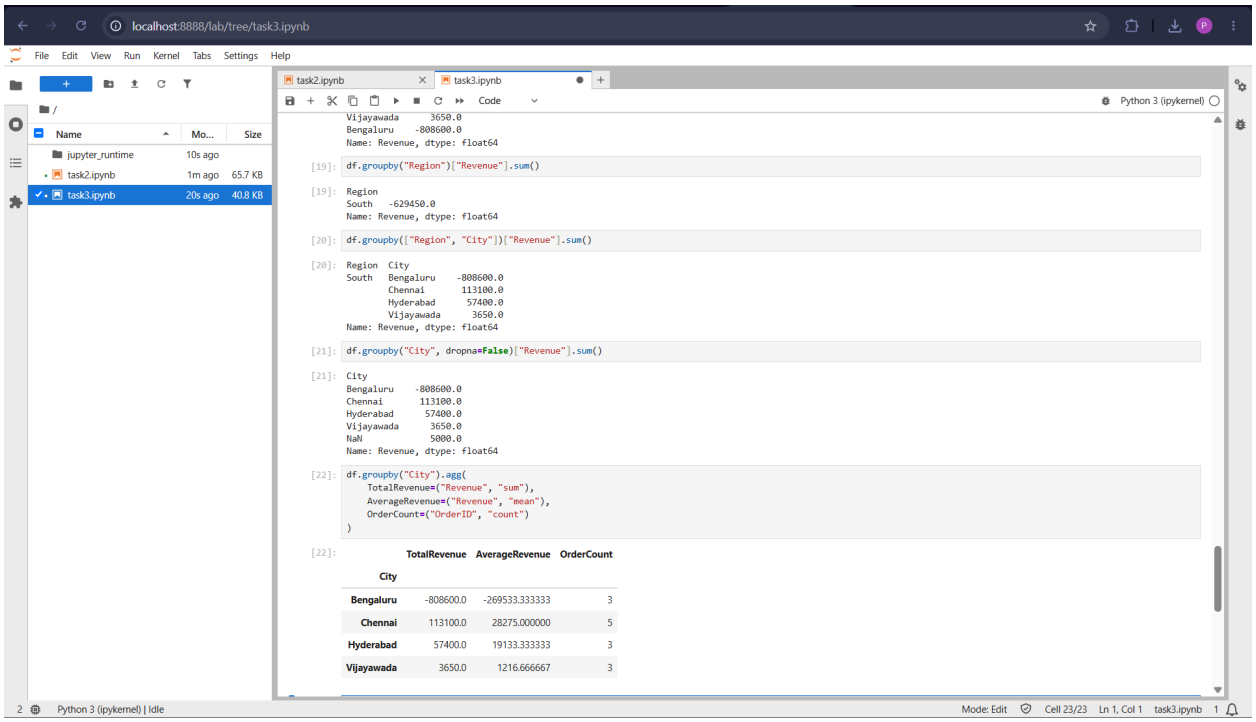


Possible causes of an unusually high price: Extra zeros, incorrect currency conversion, or an import error.

Solutions: Verify the original product price before correcting or removing the record.

Group Revenue by City





Possible causes of inaccurate revenue: Missing prices, negative quantities, duplicate orders, or missing city names.

Solutions: Verify prices and quantities, investigate repeated orders, and recover missing city information before producing final reports.

Identify Negative Quantities and Revenue

```
[23]: df[df["Quantity"] < 0]
```

[23]:

	OrderID	OrderDate	Product	Quantity	UnitPrice	City	Region	Revenue
13	ORD1014	NaT	Gaming Chair	-1	850000.0	Bengaluru	South	-850000.0

```
[24]: df[df["Quantity"] == 0]
```

[24]:

OrderID	OrderDate	Product	Quantity	UnitPrice	City	Region	Revenue

```
[25]: df[df["Revenue"] < 0]
```

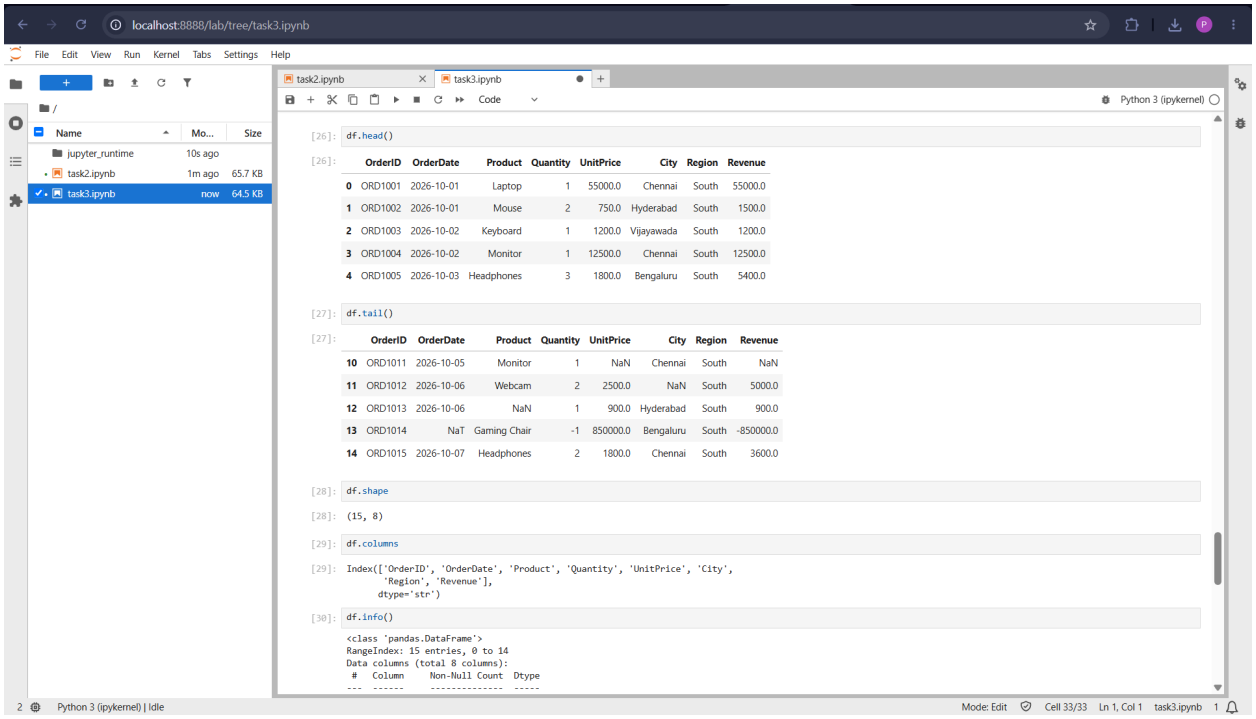
[25]:

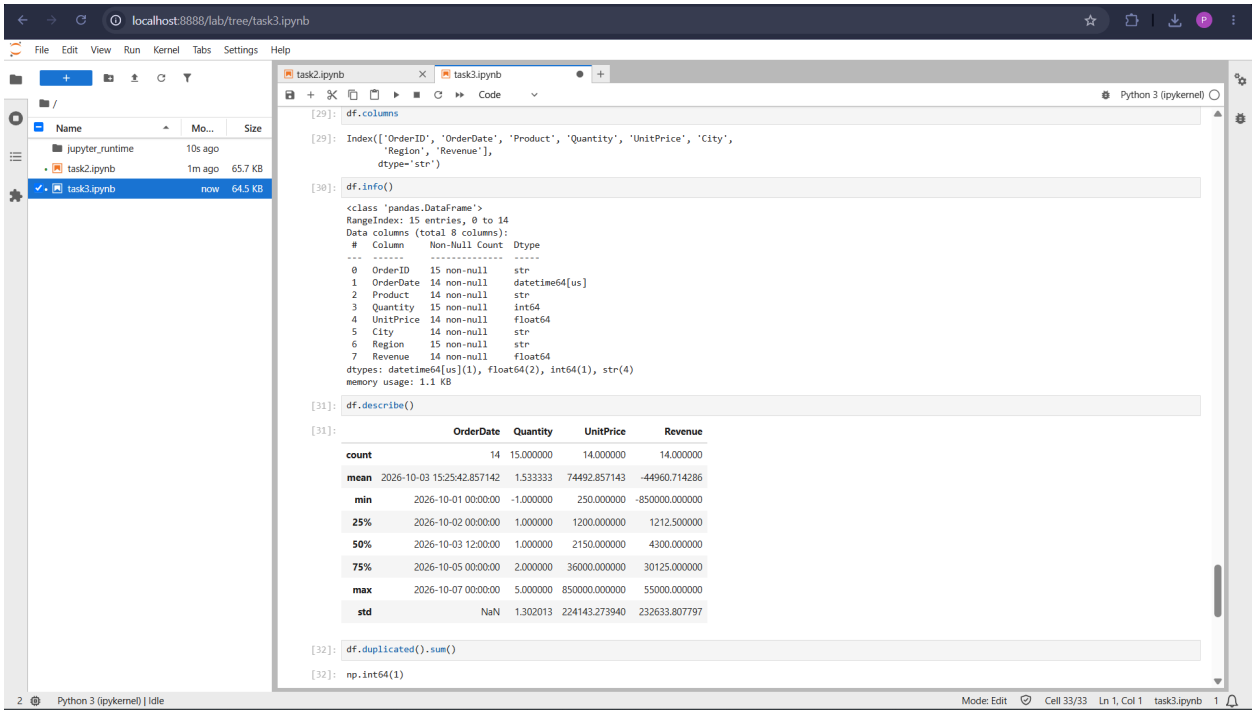
	OrderID	OrderDate	Product	Quantity	UnitPrice	City	Region	Revenue
13	ORD1014	NaT	Gaming Chair	-1	850000.0	Bengaluru	South	-850000.0

Possible causes: Data entry errors, product returns, or incorrect transaction signs.

Solutions: Verify the original transactions and distinguish returns from accidental errors.

Final Data Inspection




localhost:8888/lab/tree/task3.ipynb
File Edit View Run Kernel Tabs Settings Help
/
Name Mo... Size
jupyter_runtime 10s ago
task2.ipynb 1m ago 65.7 KB
task3.ipynb now 64.5 KB
task2.ipynb
task3.ipynb
Code
Python 3 (ipykernel)
[29]: df.columns
[29]: Index(['OrderID', 'OrderDate', 'Product', 'Quantity', 'UnitPrice', 'City', 'Region', 'Revenue'], dtype='str')
[30]: df.info()
<class 'pandas.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 8 columns):
Column Non-Null Count Dtype
0 OrderID 15 non-null str
1 OrderDate 14 non-null datetime64[us]
2 Product 14 non-null str
3 Quantity 15 non-null int64
4 UnitPrice 14 non-null float64
5 City 14 non-null str
6 Region 15 non-null str
7 Revenue 14 non-null float64
dtypes: datetime64[us](1), float64(2), int64(1), str(4)
memory usage: 1.1 KB
[31]: df.describe()
[31]:
OrderDate Quantity UnitPrice Revenue
count 14 15.000000 14.000000 14.000000
mean 2026-10-03 15:25:42.857142 1.533333 74492.857143 -44960.714286
min 2026-10-01 00:00:00 -1.000000 250.000000 -850000.000000
25% 2026-10-02 00:00:00 1.000000 1200.000000 1212.500000
50% 2026-10-03 12:00:00 1.000000 2150.000000 4300.000000
75% 2026-10-05 00:00:00 2.000000 36000.000000 30125.000000
max 2026-10-07 00:00:00 5.000000 850000.000000 55000.000000
std NaN 1.302013 224143.273940 232633.807797
[32]: df.duplicated().sum()
[32]: np.int64(1)
2 Python 3 (ipykernel) | Idle
Mode: Edit Cell 33/33 Ln 1, Col 1 task3.ipynb 1